VOL. 33

Cloud (Azure)

Accounts, resources, services and the design habits that keep an Azure solution simple, cheap and reliable

Source	microsoft.com/learn/azure
Generated	2026-10-10
Level	Beginner
Estimated study time	20 hours
Language	English
Volume	Vol. 33 - Cloud (Azure)

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 3

Chapter map 3

Chapter 1 - Cloud concepts and the shape of Azure 4

1.1 Summary 4

1.2 Concepts 5

Three ways to rent a computer 5

The shared responsibility model 5

The hierarchy of everything 6

Regions, availability zones and pairs 6

The control plane and its clients 6

1.3 Examples 7

1.4 Common mistakes and best practices 9

1.5 Practice exercises 10

Solutions 10

1.6 Mini test 10

Answer key and explanations 11

How to use this volume

Prerequisites. A terminal and either Python 3 or Node.js installed, which is all the local examples need. No Azure subscription is required to read the volume, and every command that would talk to Azure is labelled as a transcript because no subscription and no network were reachable while it was written.

Estimated study time. About 20 hours: one to two hours per chapter plus the time you spend typing the examples yourself. Read the chapters in order the first time: identity before permissions, permissions before resources, resources before networking and databases.

How the chapters work. Every chapter opens with a summary, explains the concepts with diagrams and tables, shows numbered examples with the output they produce, lists the common mistakes with the best practice for each, and ends with practice exercises, their solutions and a mini test with an answer key.

How to run the code. Blocks tagged `py` are Python scripts you run with `python file.py`; blocks tagged `js` are Node scripts you run with `node file.js`; blocks tagged `json`, `yaml`, `bicep`, `sql` or `powershell` are documents and templates you keep in a file; blocks tagged `bash` or `text` show what a terminal prints. The blocks tagged `py` and `js` were really executed on this machine and the output next to them is real.

> Important. Command line blocks that call the Azure CLI or the portal are marked not executed. Their text is a plausible transcript written for teaching, so treat names, identifiers and output as examples rather than as a recording of a real tenant.

shell

```
python --version
node --version
```

output

```
Python 3.13.15
v22.21.0
```

About the source. The syllabus follows the Azure documentation hub and the ordered learning path published on Microsoft Learn for Azure: cloud concepts first, then the fundamentals of the platform, then the architecture and adoption guidance. The service names used here are the current ones, so the identity chapter says Microsoft Entra ID where older material says Azure Active Directory.

Chapter map

#	Chapter	You will learn
1	Cloud concepts and the shape of Azure	service models, regions and availability zones, the resource hierarchy, the control plane

#	Chapter	You will learn
2	Identity with Microsoft Entra ID	users, groups, service principals, managed identities, tokens and conditional access
3	Subscriptions, resource groups and RBAC	scopes, role definitions, assignments and effective permissions
4	Compute	virtual machines, App Service, Functions and Container Apps
5	Storage accounts	blobs, files and queues, redundancy, access tiers and shared access signatures
6	Networking	virtual networks, subnets, network security groups and load balancers
7	Databases	Azure SQL Database, Azure Cosmos DB and the managed open source options
8	Integration	Service Bus queues and topics, Event Grid, Event Hubs and Logic Apps
9	Monitoring and logging	Azure Monitor, Log Analytics, Kusto queries, alerts and Application Insights
10	Governance	Azure Policy, blueprint-style governance, tags, budgets and Key Vault
11	Deploying with infrastructure as code	Bicep templates, parameters, what-if and a deployment pipeline
12	Architecting reliable solutions	the Well-Architected Framework, composite service agreements and resilience patterns
-	Cheat sheet	compact reference for the whole volume
-	Final test	25 questions with answers and explanations

Chapter 1 - Cloud concepts and the shape of Azure

1.1 Summary

- Cloud computing rents computing capacity that you start, stop and scale on demand, and you pay for what you consume instead of for hardware you own.
- Infrastructure as a service gives you a rented machine with your choice of operating system; platform as a service gives you a deploy target and no machine; software as a service gives you an application.
- The shared responsibility model splits the work: the provider secures the platform and the physical estate, and you secure identities, data and the configuration of everything you create.

- Azure arranges resources in a hierarchy: one tenant, optional management groups, subscriptions, resource groups and finally the resources themselves.
- A resource is anything a provider namespace can create, from a virtual machine to a storage account to a single tag.
- Every resource carries a resource identifier that names its subscription, resource group, provider namespace, type and name, and that same string is the scope used for permissions, policy and billing.
- Regions are geographic locations that hold one or more datacentres; availability zones are separate datacentres inside a region with their own power, cooling and network.
- Azure Resource Manager is the single control plane, which is why the portal, the Azure CLI and Azure PowerShell all do the same work with the same result.
- A subscription is a billing, quota and policy boundary, and most organisations hold several of them, for example production, development and a place for shared services.
- Cost control is a design activity: right sizing, shutdown schedules, reservations, savings plans and licence benefit change the bill far more than any reporting script can.

1.2 Concepts

Three ways to rent a computer

Model	What you get	What you still manage	Azure examples
Infrastructure as a service	a virtual machine, a network and disks	the operating system, patches, runtimes, your code	Virtual Machines
Platform as a service	a runtime or a service and a deploy target	your code, its configuration and its data	App Service, Functions, Azure SQL Database
Software as a service	an application and a login	your data and your access policy	Microsoft 365

The further down that table a service sits, the less there is to patch and the less you can tune. A virtual machine lets you install anything and leaves every patches decision to you; App Service asks for an application package and decides the operating system for you; a managed database gives you a connection string and no server at all.

The shared responsibility model

```
you own        identity, data, application code, configuration,
               network rules and every right you hand out
-------------------------------------------------------------
the provider    physical hosts, cooling, the physical network,
                the control plane and most of the hypervisor
```

Move a workload from a virtual machine to a managed database and the line moves with it: patching the engine moves from the second row to the first. The line never disappears, because somebody always owns identity and data.

The hierarchy of everything

```
tenant                       one Microsoft Entra ID directory
  |
  +- management group        optional, a place to hang policy and quota
  |    |
  |    +- subscription       billing, quota and policy boundary
  |    |    |
  |    |    +- resource group  a container for one lifecycle
  |    |         |
  |    |         +- resource  one thing a provider namespace created
  |    |         +- resource
  |    |
  |    +- subscription
  |
  +- management group        for example a sandbox with strict limits
```

Level	What it decides	Typical number
Tenant	who exists at all, and which directory a sign-in lands in	one per company, sometimes more
Management group	policy, quota and role inheritance for a group of subscriptions	a handful
Subscription	the bill, the service limits and where a budget stops	tens
Resource group	lifecycle, permissions, tags and deployment scope	hundreds
Resource	the thing itself	thousands

A resource group is a lifecycle boundary first and a folder second: deleting the group deletes everything inside it, so a resource belongs with the resources it is deployed and retired with.

Regions, availability zones and pairs

```
region, for example West Europe
  |
  +- availability zone 1   separate building, its own power and cooling
  +- availability zone 2   separate building, its own network
  +- availability zone 3   separate building, its own cooling

paired region, for example North Europe
  planned maintenance is applied to the pair one region at a time
```

Zones protect against the failure of one building. Region pairs protect against the failure of a whole region, and they are also used to stage platform updates so a paired region is not patched at the same moment.

The control plane and its clients

Azure Resource Manager receives every create, read, update and delete. Four clients speak to it, and they are interchangeable for anything that has an API:

Client	Best for	Notes
Azure portal	exploring, one-off settings, billing screens	good for reading, poor for repeating
Azure CLI	scripts, containers and any machine with a shell	`az` with subcommands per service
Azure PowerShell	Windows and PowerShell shops, module based automation	`Az` with cmdlets and a pipeline
Cloud Shell	both of the above without installing anything	runs in a browser, needs storage for the home directory

Because all four call the same control plane, anything you can click you can also script, and anything you script can be reviewed.

1.3 Examples

Example 1: what does one small virtual machine cost for three years?

Python

```python
# Pay-as-you-go compared with a reservation for one small Linux VM.
months = 36
hours_per_month = 730

vm_hourly = 0.0464
disk_monthly = 2.30
egress_monthly = 1.15

monthly = vm_hourly * hours_per_month + disk_monthly + egress_monthly
reserved_per_year = 260.00
reserved = 2 * reserved_per_year + (disk_monthly + egress_monthly) * months
on_demand = monthly * months

print("period            : %d months" % months)
print("pay as you go     : %.2f per month" % monthly)
print("pay as you go     : %.2f for %d months" % (on_demand, months))
print("two reservations  : %.2f (disk and egress stay hourly)" % reserved)
print("saving            : %.2f, which is %.1f%% of the bill"
      % (on_demand - reserved, 100 * (on_demand - reserved) / on_demand))
print("one day of compute: %.2f, a stopped VM stops the compute charge"
      % (vm_hourly * 24))
```

output

```
period            : 36 months
pay as you go     : 37.32 per month
pay as you go     : 1343.59 for 36 months
two reservations  : 644.20 (disk and egress stay hourly)
saving            : 699.39, which is 52.1% of the bill
one day of compute: 1.11, a stopped VM stops the compute charge
```

The prices are illustrative round numbers that stand in for a real quote from the pricing calculator; the arithmetic and the shape of the answer are what matter. A reservation only pays for itself when the workload really runs for a year or three years, and a stopped virtual machine stops billing for compute while its disks keep costing money.

Example 2: create a resource group with the Azure CLI. Not executed, because no subscription is reachable from this machine.

shell

```
az group create --name rg-shop-dev --location westeurope
```

JSON

```
{
  "id": "/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/rg-shop-dev",
  "location": "westeurope",
  "managedBy": null,
  "name": "rg-shop-dev",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
```

Three things happen here that repeat for almost every command: the name is lower case and globally unique for resources that need a public endpoint, the location is chosen once and rarely changes, and the answer includes the resource identifier that everything else refers to.

Example 3: read a resource identifier into its parts.

Python

```
# Pull an Azure resource identifier apart into its scope levels.
rid = ("/subscriptions/00000000-0000-0000-0000-000000000000"
       "/resourceGroups/rg-shop-dev"
       "/providers/Microsoft.Storage/storageAccounts/stshopdev01"
       "/blobServices/default/containers/images")

tokens = rid.strip("/").split("/")
rows = []
i = 0
while i < len(tokens):
    key = tokens[i].lower()
    if key == "subscriptions":
        rows.append(("subscription", tokens[i + 1]))
        i += 2
    elif key == "resourcegroups":
        rows.append(("resource group", tokens[i + 1]))
        i += 2
    elif key == "providers":
        rows.append(("provider", tokens[i + 1]))
        i += 2
        while i + 1 < len(tokens) and tokens[i].lower() != "providers":
            rows.append((tokens[i] + " name", tokens[i + 1]))
            i += 2
    else:
        i += 1

for label, value in rows:
    print("%-22s %s" % (label, value))

print()
print("total tokens     : %d" % len(tokens))
print("resource group id: /subscriptions/%s/resourceGroups/%s"
      % (rows[0][1], rows[1][1]))
```

output

```
subscription           00000000-0000-0000-0000-000000000000
resource group         rg-shop-dev
provider               Microsoft.Storage
storageAccounts name   stshopdev01
blobServices name      default
containers name        images

total tokens     : 12
resource group id: /subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/rg-shop-dev
```

Child resources continue the same pattern: after the provider namespace the string alternates between a type and a name, so `storageAccounts/stshopdev01` means "the resource of type storageAccounts called stshopdev01". The prefix of this string, up to and including the resource group, is the scope used for permissions and policy.

Example 4: show which directory and subscription a shell is pointed at. Not executed.

shell

```
az account list --output table
```

output

```
Name            CloudName    SubscriptionId                        TenantId
Visual Studio   AzureCloud   aaaa5a5a-9a5a-4a5a-8a5a-a5a5a5a5a5a5  11111111-1111-1111-1111-111111111111
pay-as-you-go   AzureCloud   00000000-0000-0000-0000-000000000000  11111111-1111-1111-1111-111111111111
```

The combination of tenant and subscription decides what you can see. A person with two tenants on one shell must switch between them before a command means anything, and `az account set --subscription` is the command that does it.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Treating a resource group as a folder	resources are retired with the group, so a shared database disappears with an experiment	group by lifecycle, then by owner
Putting production and development in one subscription	quotas, budgets and policy bleed across environments	one subscription per environment, one management group for both
Picking a region per resource instead of per solution	data movement between regions is slow and metered	decide the region once, deploy the whole solution there
Ignoring availability zones until the first outage	a zone failure takes a whole tier down	plan zone redundancy for anything that serves traffic
Reading cost reports after the bill arrives	the decision was made months earlier at design time	estimate with the pricing calculator before you create anything

Mistake	Why it hurts	Best practice
Doing everything in the portal	nothing is repeatable and nothing is reviewable	click to explore, script to build, keep both in step
Assuming a stopped virtual machine stops the bill	disks and public addresses keep billing	deallocate, or delete the whole resource group for short lived tests

1.5 Practice exercises

1. Classify each service as infrastructure, platform or software as a service: Virtual Machines, App Service, Azure SQL Database, Azure Cosmos DB, Functions and Microsoft 365.
2. A storage account identifier starts with `/subscriptions/1234.../resourceGroups/rg-analytics/providers/Microsoft.Storage/storageAccounts/stan01`. Which subscription and resource group own it, and which provider namespace created it?
3. A development virtual machine is only needed from nine in the morning to six in the evening on weekdays. Using the numbers from Example 1, what fraction of the compute bill does that schedule save?
4. Why does it matter that the portal, the Azure CLI and Azure PowerShell all go through the same control plane when you write a runbook for an operations team?

Solutions

1. Virtual Machines is infrastructure as a service because you receive the operating system and patch it yourself. App Service, Azure SQL Database, Azure Cosmos DB and Functions are platform as a service because you hand over code or a schema and receive a managed runtime. Microsoft 365 is software as a service because you receive a working application and only manage your data.
2. The subscription is `1234...`, the resource group is `rg-analytics` and the provider namespace is `Microsoft.Storage`. The scope used for permissions on this account is the prefix up to and including the resource group.
3. Nine hours a day for five days a week is 45 hours out of the 168 in a week, which is about 27 percent. Running the machine only then saves roughly 73 percent of the compute charge, about 25 of the 37.32 per month in Example 1, while disks continue to bill.
4. Because an operation recorded in the portal is the same operation the CLI performs, a runbook can start as a click, be copied from the portal history into a script, and then be reviewed, parameterised and run unattended. If the two clients behaved differently the runbook would drift from reality.

1.6 Mini test

1. Which service model hands you an operating system you must patch yourself? - A) Platform as a service - B) Infrastructure as a service - C) Software as a service - D) Serverless

2. True or false: a resource group is only a visual folder, so deleting it has no effect on the resources inside it.
3. Which level of the hierarchy holds the bill and the service quota?
4. What does the string `providers/Microsoft.Web/sites` in a resource identifier tell you?
5. True or false: two resources in the same subscription can live in different regions.
6. Why do availability zones and region pairs protect against different failures?
7. Which Azure client would you choose for a command that must run unattended every night?

Answer key and explanations

1. B - infrastructure as a service rents the machine and leaves the operating system, its patches and everything installed on it to you.
2. False - a resource group is a lifecycle boundary. Deleting it deletes every resource inside, which is why shared resources get their own group.
3. The subscription. It is the billing and quota boundary, and it is where a budget is attached in Cost Management.
4. That the resource belongs to the `Microsoft.Web` provider namespace and its type is `sites`, which is the type behind an App Service app.
5. True - a subscription spans every region, so a solution can deploy one tier in one region and another tier elsewhere when there is a reason to.
6. Availability zones are separate buildings inside one region and protect against the loss of a building or its power and cooling. A region pair is a different region a long way away and protects against the loss of the whole region.
7. The Azure CLI or Azure PowerShell, because both are non interactive and can be scheduled. The portal is fine for reading, and Cloud Shell is fine for one-off work but it is not a scheduler.